Procesos de Desarrollo de Software

CLASE 3 – BAD SMELLS / GRASP

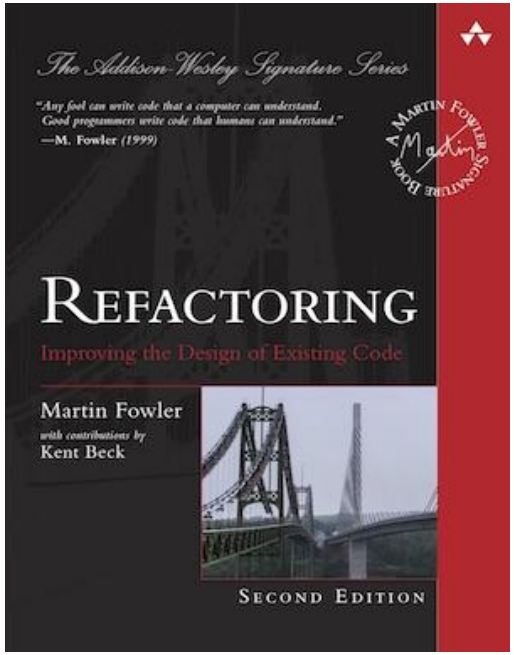
Agenda de la clase

- Bad Smell
- GRASP

Bad smells

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

– Martin Fowler 1999



Bad smell

- Los smells no necesariamente significan que esté mal lo que diseñé/implementé, sino que son señales de algo que se puede mejorar.
  - No hay que ser terminante, muchos code smells se entrecruzan al tratar de solucionarlos
- La idea siempre es mejorar nuestro diseño.

Ejemplo

```
public void trotar(int metros){
    this.energía = this.energía - 10;
    this.animo = this.animo - 3;
    this.posición = new Poi(this.posición.getX()+metros, this.posición.getY()+metros);
}

public void caminar(int metros){
    this.energía = this.energía - 1;
    this.animo = this.animo -1;
    this.posición = new Poi(this.posición.getX()+metros, this.posición.getY()+metros);
}
```

Refactorizando

```
public void trotar(int metros){
    this.disminuirEnergía(10);
    this.disminuirAnimo(3);
    this.desplazar(metros);
}

public void caminar(int metros){
    this.disminuirEnergía(1);
    this.disminuirAnimo(1);
    this.desplazar(metros);
}
```

```
public void trotar(int metros){
    mover(metros, 10, 3);
}

public void caminar(int metros){
    mover(metros, 1, 1);
}
```

Usando objetos

```
interface Movimiento {
    void mover(Personaje p, int metros);
}

class Caminar implements Movimiento {
    public void mover(Personaje p, int metros){
        p.restarEnergia(1);
        p.restarAnimo(1);
        p.moverPosicion(metros);
    }
}

class Trotar implements Movimiento {
    public void mover(Personaje p, int metros){
        p.restarEnergia(10);
        p.restarAnimo(3);
        p.moverPosicion(metros);
    }
}
```

Duplicación de código

- No queremos duplicar la misma idea en el código.

Once and only once: hacer las cosas una sola vez

Don't repeat yourself (DRY)

- Y esto no sólo vale para el código, también aplica para el diseño en general. Dos de las formas más conocidas para evitar este smell en diseño es la composición y la herencia.

Clasificación

Group name	Smells in group
The Bloaters *Inflados*	-Long Method -Large Class -Primitive Obsession -Long Parameter List -DataClumps
The Object-Orientation Abusers *Abusando de la POO*	-Switch Statements -Temporary Field -Refused Bequest -Alternative Classes with Different Interfaces
The Change Preventers *Frustradores del Cambio*	-Divergent Change -Shotgun Surgery -Parallel Inheritance Hierarchies
The Dispensables *Prescindibles*	-Lazy class -Data class -Duplicate Code -Dead Code, -Speculative Generality
The Couplers *Acopladores*	-Feature Envy -Inappropriate Intimacy -Message Chains -Middle Man

Bad smells in code

- Es una **clasificación de problemas de diseño** en el código (code smells).
- No son errores de compilación
- Son **malas decisiones de diseño** que hacen al sistema:
  - difícil de mantener
  - difícil de extender
  - propenso a bugs

The Bloaters (Clases infladas)

- Cosas que crecen demasiado

Problemas:

- **Large Class** → clase gigante (God Class)
- **Long Method** → métodos larguísimos
- **Primitive Obsession** → usar int, String en lugar de objetos
- **Long Parameter List** → muchos parámetros
- **Data Clumps** → datos que siempre viajan juntos

The Object-Orientation Abusers (abuso de la OO)

- Mal uso de la orientación a objetos

Problemas:

- **Switch Statements** → muchos if o switch
- **Temporary Field** → atributos que se usan a veces
- **Refused Bequest** → herencia mal usada
- **Alternative Classes with Different Interfaces** → clases parecidas con métodos distintos

The Change Preventers (desalentadores del cambio)

- Cosas que hacen difícil cambiar el sistema

Problemas:

- **Divergent Change** → una clase cambia por muchas razones
- **Shotgun Surgery** → cambiar una cosa implica tocar mil clases
- **Parallel Inheritance Hierarchies** → una nueva jerarquía afecta otras en paralelo

The Dispensables (prescindibles)

- Código innecesario

Problemas:

- **Lazy Class** → clase inútil
- **Duplicate Code** → código repetido
- **Dead Code** → código muerto
- **Speculative Generality** → sobreingeniería

The Couplers (acoplados)

- Problemas de acoplamiento

Problemas:

- **Feature Envy** → una clase usa más datos de otra
- **Inappropriate Intimacy** → clases demasiado dependientes
- **Message Chains** → a.getB().getC().getD()
- **Middle Man** → clase que solo delega

God class (large class)

- Las clases por lo general comienzan siendo pequeñas, pero con el tiempo se hinchan a medida que el Sistema crece. Es decir, la clase hace "**de todo**".

- Para solucionar esto debemos:
  - **Generar nuevas clases**: si el comportamiento de la clase es grande, y cumple con varias funciones al mismo tiempo, se debe dividir la God Class en más clases que cumplan con un solo propósito.
  - **Generar subclases**: si parte del comportamiento de la God Class se puede implementar de diferentes maneras, entonces una solución es convertir a nuestra vieja God Class en una clase Abstracta y luego generar subclases que definan el comportamiento específico necesario.
  - **Generar interfaces**: si lo que queremos es brindar una lista de operaciones con diferentes comportamientos que el cliente puede utilizar, podemos generar una/varias interface(s) y luego crear clases concretas que cumplan con dicha interfaz e implementen las diferentes funcionalidades.

Parámetros largos

- Puede darse cuando varios algoritmos se combinan en un solo método. O también cuando nos estamos olvidando de alguna abstracción del dominio.
- Si un objeto va a brindar un servicio a otro, lo mejor sería que la cantidad de parámetros del método en cuestión no varíe.

```
public void crearPedido(
    String nombreCliente,
    String direccion,
    String telefono,
    String email,
    String producto,
    int cantidad,
    double precio
) {
    System.out.println("Pedido creado");
}
```

```
crearPedido(
    "Juan",
    "Av Siempre Viva 123",
    "123456",
    "juan@mail.com",
    "Pizza",
    2,
    1500
);
```

Parámetros largos: problemas

- No se entiende qué es cada cosa
- Fácil equivocarse

```
crearPedido("Juan", "Pizza", "juan@mail.com", "123456", "Av Siempre Viva 123", 1500, 2);
```

- Difícil mantenimiento: si agrego, por ejemplo el código postal, debo cambiar el método y **todas** las llamadas, tests, servicios.

Solución

```java
public class Cliente {
    String nombre;
    String direccion;
    String telefono;
    String email;
}

public class ItemPedido {
    String producto;
    int cantidad;
    double precio;
}

public void crearPedido(Cliente cliente, ItemPedido item) {
    System.out.println("Pedido creado");
}
```

```java
Cliente c = new Cliente();
ItemPedido item = new ItemPedido();

crearPedido(c, item);
```

```java
Pedido pedido = new PedidoBuilder()
    .nombre("Juan")
    .direccion("Av Siempre Viva")
    .telefono("123")
    .producto("Pizza")
    .cantidad(2)
    .build();
```

Ruptura de encapsulamiento

- Rompe el principio:
  - Los atributos deben estar protegidos y modificarse sólo mediante comportamiento.

```java
public class CuentaBancaria {
    public double saldo;
}

public class Main {
    public static void main(String[] args) {

        CuentaBancaria cuenta = new CuentaBancaria();

        cuenta.saldo = 1000;
        cuenta.saldo = -500;   // ❌ inválido
        cuenta.saldo = 999999; // ❌ cualquiera puede cambiarlo
    }
}
```

Solución

```java
public class CuentaBancaria {

    private double saldo;

    public void depositar(double monto) {
        saldo += monto;
    }

    public void extraer(double monto) {
        if (monto <= saldo) {
            saldo -= monto;
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

```java
CuentaBancaria cuenta = new CuentaBancaria();

cuenta.depositar(1000);
cuenta.extraer(500);

System.out.println(cuenta.getSaldo());
```

Ruptura de encapsulamiento

- Otro momento es se presenta cuando una clase tiene dependencias en la implementación de otra clase.
- Por ejemplo: sabemos que una clase tiene una lista de objetos, y que a su vez esos objetos tienen una lista de otros objetos, y así sucesivamente…
  - Algo del estilo:

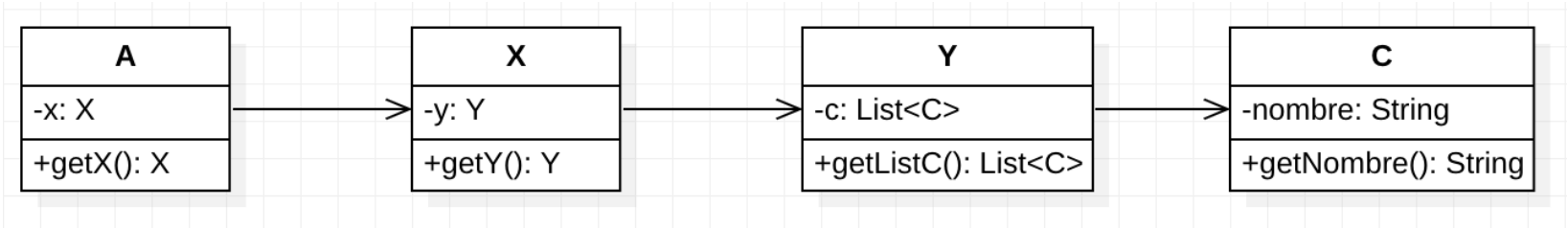
```
a.getX().getY().getListC().….
```

- **Solución**: hablar y pedir.

Ruptura de encapsulamiento

- Significa que una clase está **navegando demasiado la estructura interna de otras**.

```
a.getX().getY().getListC().get(0).getNombre();
```



Herencia rechazada

- Se da cuando una clase sobrescribe completamente un método base definido por su padre sin importarle lo que éste le haya dicho que tenía que hacer.
- La herencia es rígida:
  - Obliga a definir más métodos que los necesarios (en ciertos casos)
- **Solución**: si no son lo mismo, ¿para qué forzarlo? ***Utilizar composición por sobre herencia.***

Herencia rechazada

```java
class Ave {
    public void volar() {
        System.out.println("Volando...");
    }
}

class Paloma extends Ave {
}

class Pinguino extends Ave {
    @Override
    public void volar() {
        throw new UnsupportedOperationException("No puedo volar");
    }
}
```

- Pinguino **no debería volar**, pero está obligado a heredar volar()

Herencia rechazada

```java
class Motor {
    public void encender() {
        System.out.println("Motor encendido");
    }
}

class Auto extends Motor {
    public void abrirPuertas() {
        System.out.println("Puertas abiertas");
    }
}

Auto auto = new Auto();
auto.encender();
```

- Funciona, sí, pero conceptualmente está mal modelado. Auto **NO** es un motor.
- El problema no es sólo técnico: **el dominio está mal expresado**.

Solución

```java
class Motor {
    public void encender() {
        System.out.println("Motor encendido");
    }
}

class Auto {
    private Motor motor;

    public Auto(Motor motor) {
        this.motor = motor;
    }

    public void encenderAuto() {
        motor.encender();
    }

    public void abrirPuertas() {
        System.out.println("Puertas abiertas");
    }
}
```

Lazy class

- La clase no hace **nada** o hace **muy poco**.
  - tiene pocos métodos
  - no tiene comportamiento
  - sólo delega
  - no aporta valor
- Suele darse en el caso de la extrema delegación o porque el componente fue diseñado "**por las dudas**".

Ejemplos

```java
class Direccion {
    private String calle;

    public Direccion(String calle) {
        this.calle = calle;
    }

    public String getCalle() {
        return calle;
    }
}

class Cliente {
    private Direccion direccion;

    public Cliente(Direccion direccion) {
        this.direccion = direccion;
    }

    public String obtenerCalle() {
        return direccion.getCalle();
    }
}
```



```java
class Cliente {
    private String calle;

    public Cliente(String calle) {
        this.calle = calle;
    }

    public String obtenerCalle() {
        return calle;
    }
}
```

Ejemplos

```java
class Calculadora {
    public int sumar(int a, int b) {
        return a + b;
    }
}

class Servicio {
    private Calculadora calculadora = new Calculadora();

    public int calcular() {
        return calculadora.sumar(2, 3);
    }
}
```



```java
class Servicio {
    public int calcular() {
        return 2 + 3;
    }
}
```

```java
class PedidoManager {

    private Pedido pedido;

    public PedidoManager(Pedido pedido) {
        this.pedido = pedido;
    }

    public void confirmar() {
        pedido.confirmar();
    }
}
```

Complejidad artificial

- Se presenta con el uso forzado de patrones de diseño muy complicados, donde uno más simple sería suficiente.
- Es decir, el diseño es **más complicado de lo necesario sin una razón real**.
  - demasiadas clases
  - demasiada abstracción
  - interfaces innecesarias
  - patrones sin necesidad
- ***Solución***: usar lo que sabemos cuando es realmente necesario y no "por las dudas".

Ejemplo

```
class CalculadoraFactory {

    public static Calculadora crearCalculadora() {
        return new Calculadora();
    }
}

class Calculadora {
    public int sumar(int a, int b) {
        return a + b;
    }
}
```

```
Calculadora c = CalculadoraFactory.crearCalculadora();
c.sumar(2,3);
```

```
Calculadora c = new Calculadora();
c.sumar(2,3);
```

Nombres de variables muy cortos

- No expresa intención
- El nombre de la variable debe representar su funcionalidad → ***expresividad***.

```
int x = 10;
int y = 5;
int z = x * y;
```


x ¿precio?
y ¿cantidad?
z ¿total?

```
int precio = 10;
int cantidad = 5;
int total = precio * cantidad;
```

Nombre de métodos muy largos

- Si nos está costando escoger un nombre bien representativo para el método, y nos queda muy largo, seguramente hay más de una lógica comprendida dentro del mismo.

```
public void calcularTotalDelPedidoAplicandoDescuentoYSumandoImpuestosYValidandoStockDisponible(){...}
```

- Problemas:
  - demasiado largo
  - difícil de leer
  - hace demasiadas cosas
  - probablemente God Method



```
public void calcularTotal(){...}
public void aplicarDescuento(){...}
public void aplicarImpuestos(){...}
public void validarStock(){...}

public void procesarPedido() {
    validarStock();
    calcularTotal();
    aplicarDescuento();
    aplicarImpuestos();
}
```

Código muerto

- El típico código "***por las dudas***" que nos marea a la hora de ver la responsabilidad que tiene asignada una clase.
  - métodos que no se usan
  - responsabilidad poco clara
  - ruido en la clase
  - confunde al lector
  - complejidad innecesaria
- **Solución**: Borrar todo el code muerto y, otra vez, diseñar de modo extensible pero cerrado al cambio.

Ejemplo

```java
public class Pedido {

    private double total;

    public void confirmar() {
        System.out.println("Pedido confirmado");
    }

    // por si en el futuro hay envío internacional
    public void calcularImpuestosInternacionales() {
        System.out.println("Calculando impuestos internacionales");
    }

    // por si hay promociones especiales
    public void aplicarPromocionEspecial() {
        System.out.println("Aplicando promo especial");
    }

    // por si agregamos criptomonedas
    public void pagarConCripto() {
        System.out.println("Pago con cripto");
    }
}
```

No diseñar para cosas que **tal vez** ocurran.

Esto viola:
YAGNI — *You Ain't Gonna Need It*

Primitive obsession

- Ocurre cuando usamos **tipos primitivos (string, int, double, etc.)** para representar conceptos del dominio que deberían ser **objetos**.
- Eso hace que:
  - el código sea confuso
  - haya validaciones repetidas
  - se rompa encapsulamiento
  - aparezcan muchos parámetros

Ejemplo

```java
public class Pedido {

    private String estado;
    private String fecha;
    private String emailCliente;

    public void cambiarEstado(String estado) {
        this.estado = estado;
    }
}
```



```java
public class Pedido {

    private EstadoPedido estado;
    private Fecha fecha;
    private Email emailCliente;
}
```

Problemas:

- estado es String → “ENVIADO”, “enviado”, “Env”, etc.
- fecha es String → “10/10/24” ??? “2024-10-10” ???
- email es String sin validar

Data Class - Only accessors

- Una clase que sólo representa una estructura de datos es fácil verla porque tiene sólo getters y setters.

- Es mala práctica pensar que es bueno separar un objeto en atributos y comportamiento, ya que rompe con nuestro paradigma de objetos; donde un objeto agrupa comportamiento y atributos.

- Pese a esto, hay algunos objetos que sí tienen permitido tener solamente datos: los objetos **DTO** (Data Transfer Object), por ejemplo.

Ejemplo

```
class Pedido {
    double total;
    String estado;

    //getters/setters...
}
```

- Y la lógica afuera...

```
pedido.setEstado("CONFIRMADO");
```

```
class Pedido {

    private double total;
    private String estado;

    public void confirmar(){
        this.estado = "CONFIRMADO";
    }
}
```

GRASP

DISEÑO DE OBJETOS CON RESPONSABILIDADES

GRASP

- **GRASP** = *General Responsibility Assignment Software Patterns*
- *Después de la **identificación de los requisitos** y la **creación** de un **modelo del dominio**, entonces se añade **métodos** a las **clases de software** y se define el paso de **mensajes** entre los **objetos** para satisfacer los **requisitos**.*
- Son **reglas para decidir qué clase debe hacer qué**.
  - O sea: **cómo repartir responsabilidades correctamente**.

Responsabilidades y métodos

- Es un contrato u obligación de un clasificador.
- Conocer
  - **Conocer** los objetos privados encapsulados.
  - **Conocer** los objetos relacionados.
  - **Conocer** las cosas que puede derivar o calcular
- Hacer
  - **Hacer** algo él mismo, como crear un objeto o hacer un cálculo.
  - **Iniciar** una acción entre dos objetos.
  - **Controlar** y **coordinar** actividades en otros objetos

Responsabilidades y métodos

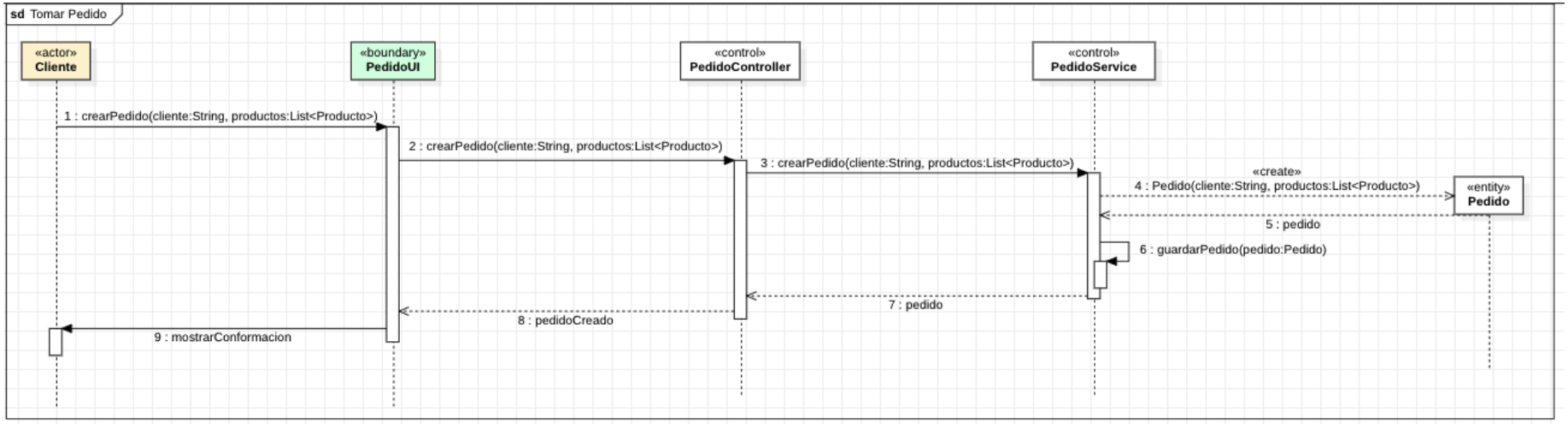
- Una responsabilidad **NO** es lo mismo que un método, pero los métodos se implementan para llevar a cabo las responsabilidades.

Ejemplos:

- Resolver los accesos a la Base de Datos.
- Crear una Venta

Responsabilidades y diagramas de interacción

- Los **diagramas de interacción** muestran elecciones en la asignación de responsabilidades a los objetos.
- Cuando los creamos, estamos tomando las **decisiones** de cómo **asignar responsabilidades** a los objetos.



Patrones

- Un **patrón** es una descripción de un **problema** y la **solución**, a la que se le da un **nombre**; y que puede ser aplicado a nuevos contextos, con consejos acerca de cómo aplicarlo en nuevas situaciones y discusiones sobre sus compromisos.
  - Nombre
  - Problema
  - Solución
- Los patrones **GRASP** describen los **principios fundamentales** del diseño de objetos y **la asignación de responsabilidades**, expresados como **patrones**.

Los 9 patrones GRASP

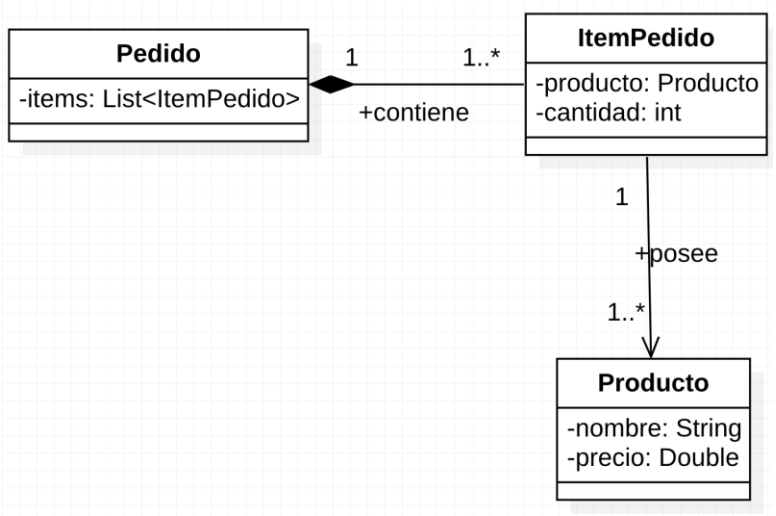
- Information Expert
- Creator
- Controller
- Low Coupling
- High Cohesion
- **Polymorphism**
- Pure Fabrication
- Indirection
- Protected Variations

Information Expert

- **Problema**: ¿Cuál es el principio general para asignar responsabilidades a los objetos?
- **Solución**: Asignar una responsabilidad al experto en información (la clase que tiene la información necesaria para realizar la responsabilidad)
- **Beneficios**:
  - Mantiene el encapsulamiento, los objetos utilizan su propia información para llevar a cabo sus tareas.
  - Se distribuye el comportamiento entre las clases que contienen la información requerida.
  - Son más fáciles de entender y mantener.

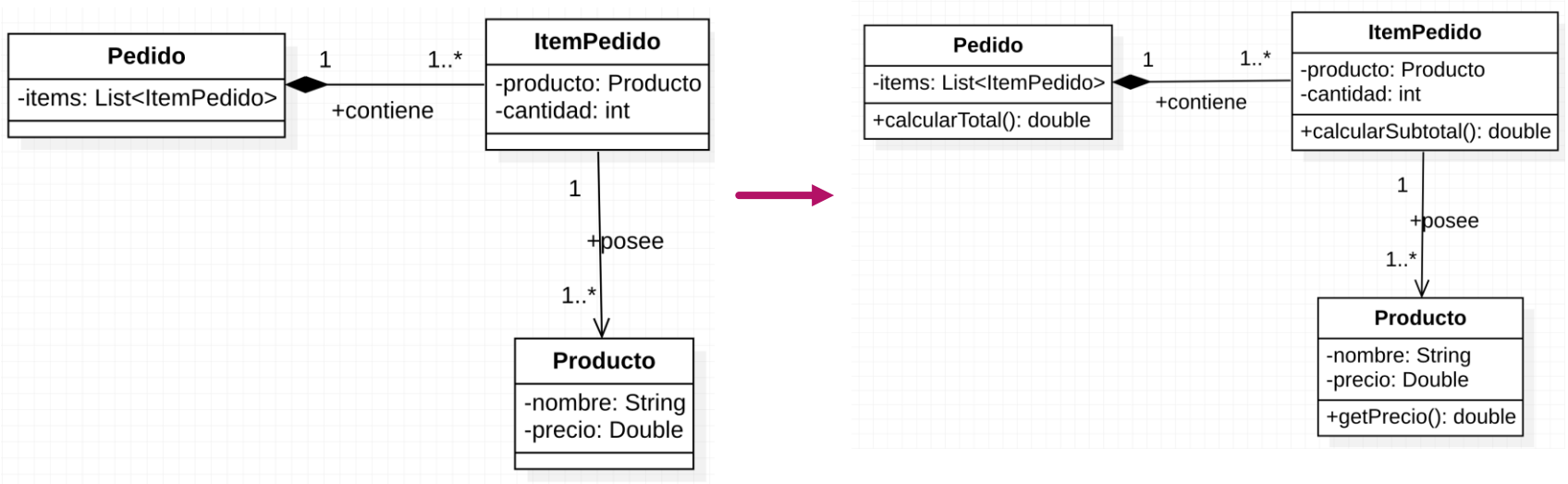
Ejemplo

- ¿Quién es el responsable de conocer el total de la venta?



Ejemplo

- **Pedido** debe **calcular** el **Total** porque posee todos los ítems
- **ItemPedido** debe **calcular** el **subtotal** porque conoce el Producto y su cantidad
- **Producto** debe **devolver** su **precio**.

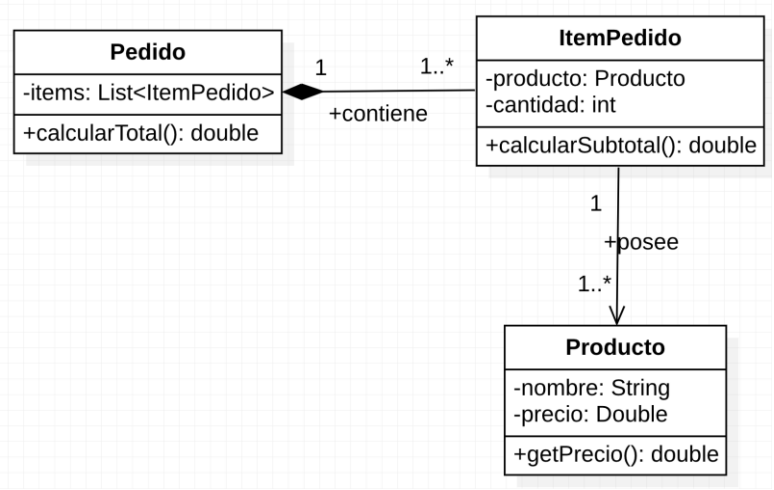


Creator

- **Problema**: ¿Quién debería ser responsable de la creación de una nueva instancia de alguna clase?
- **Solución**
- La nueva instancia deberá ser creada por la clase que:
  - Tiene la **información** necesaria para realizar la creación del objeto, o
  - Usa **directamente** las instancias creadas del objeto, o
  - **Almacena** o maneja varias instancias de la clase
  - **Contiene** o agrega la clase.
- La nueva instancia deberá ser creada por la clase que:
  - A **contiene** B
  - A **usa** B
  - A **tiene los datos para construir B**
  - A **registra** B
  - A **controla el ciclo de vida de B**

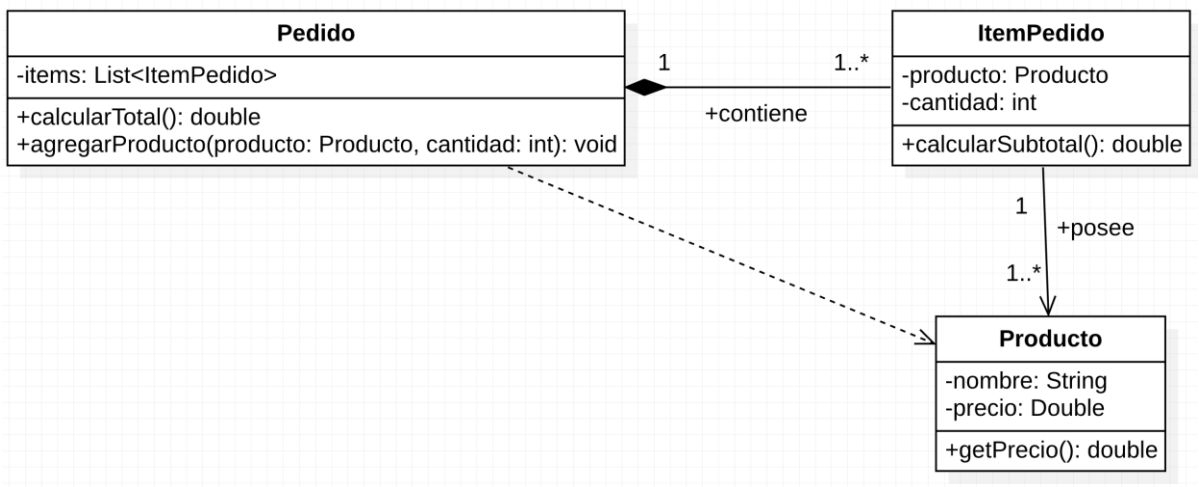
Ejemplo

- ¿Quién debe crear el **ItemPedido**?



Ejemplo

- **Pedido** crea **ItemPedido** porque:
  - lo contiene
  - lo administra
  - controla su ciclo de vida



Ejemplo

```
Producto hamburguesa = new Producto("Hamburguesa", 3500);

ItemPedido item = new ItemPedido(producto, 2);
pedido.agregarItem(item);
```

- ❌ otra clase crea ItemPedido
- ❌ Pedido no controla la creación
- ❌ rompe GRASP Creator



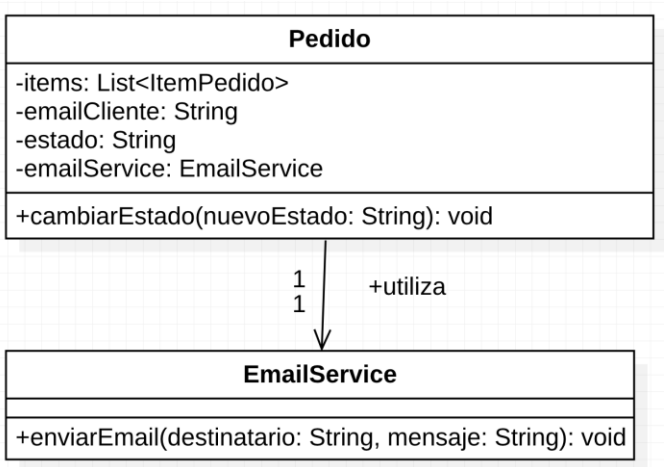
```
Pedido pedido = new Pedido();
pedido.agregarProducto(hamburguesa, 2);
```

Low coupling

- **Problema**: ¿Cómo soportar bajas dependencias, bajo impacto del cambio e incremento de la reutilización?
- **Solución**: asignar una responsabilidad de manera que el acoplamiento permanezca bajo.
- Un elemento con **bajo acoplamiento** no depende demasiado de otros elementos.
- Una clase con alto acoplamiento confía en muchas otras clases. Esto conlleva los siguientes problemas:
  - Un cambio en alguna clase relacionada fuerza a cambios locales.
  - Son difíciles de entender de manera aislada.
  - Son difíciles de reutilizar puesto que para su uso se requiere las clases adicionales relacionadas.

Ejemplo

- Supongamos que cuando cambia el estado de un pedido, queremos notificar al cliente. Una solución podría ser:




```java
class Pedido {
    private String estado;
    private String emailCliente;
    private EmailService emailService;

    public Pedido(String emailCliente) {
        this.emailCliente = emailCliente;
        this.emailService = new EmailService();
    }

    public void cambiarEstado(String nuevoEstado) {
        this.estado = nuevoEstado;
        emailService.enviarEmail(emailCliente, "El pedido cambió a estado: " + estado);
    }
}

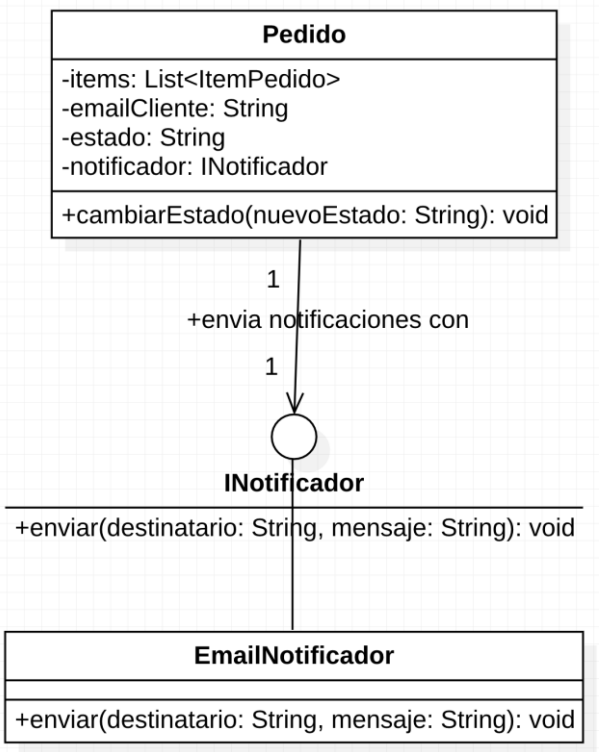
class EmailService {
    public void enviarEmail(String destinatario, String mensaje) {
        System.out.println("Enviando email a " + destinatario + ": " + mensaje);
    }
}
```

Ejemplo

- **¿Resuelve el problema?**
- Sí. Cuando cambia el estado, manda el email.
- **¿Cuál es el problema?**
- Que **Pedido** está fuertemente acoplado a **EmailService**, porque:
  - lo crea con new
  - conoce la clase concreta
  - si mañana quiero usar WhatsApp o Push Notification, debo modificar Pedido
  - si cambia la librería de email, también debo tocar Pedido

Ejemplo

- Una mejor solución sería:




```java
class Pedido {
    private String estado;
    private String contactoCliente;
    private Notificador notificador;

    public Pedido(String contactoCliente, Notificador notificador) {
        this.contactoCliente = contactoCliente;
        this.notificador = notificador;
    }

    public void cambiarEstado(String nuevoEstado) {
        this.estado = nuevoEstado;
        notificador.enviar(contactoCliente, "El pedido cambió a estado: " + estado);
    }
}

interface Notificador {
    void enviar(String destinatario, String mensaje);
}

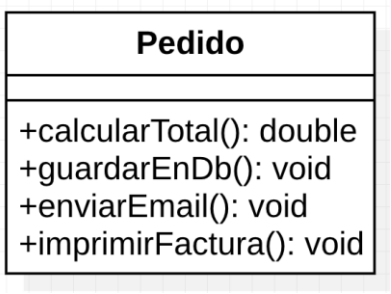
class EmailNotificador implements Notificador {
    @Override
    public void enviar(String destinatario, String mensaje) {
        System.out.println("Enviando email a " + destinatario + ": " + mensaje);
    }
}
```

High Cohesion

- **Problema**: ¿Cómo mantener la complejidad manejable?
- **Solución**: Asignar una responsabilidad de manera que la cohesión permanezca alta.
- Un elemento con responsabilidades altamente relacionadas, tiene **alta cohesión**.
- Una clase con **baja cohesión** hace muchas cosas no relacionadas, o hace demasiado trabajo, lo que lleva a los siguientes problemas:
  - Difíciles de entender.
  - Difíciles de reutilizar.
  - Difíciles de mantener.
  - Delicadas, constantemente afectadas por los cambios.

Ejemplo

- Pedido:
  - hace lógica de negocio
  - acceso a datos
  - notificaciones
  - impresión




```java
class Pedido {

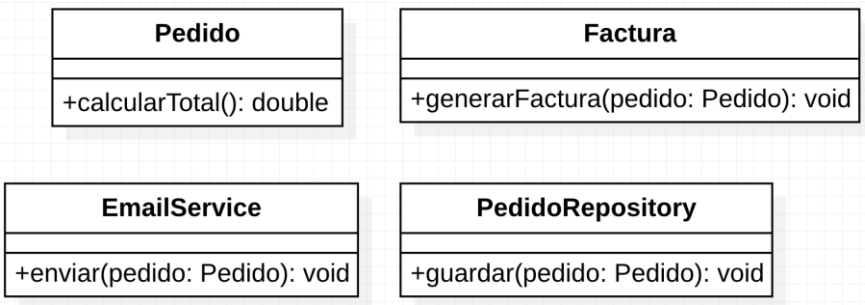
    public double calcularTotal() {
        return 100;
    }

    public void guardarEnDB() {
        System.out.println("Guardando en DB");
    }

    public void enviarEmail() {
        System.out.println("Enviando email");
    }

    public void imprimirFactura() {
        System.out.println("Imprimiendo factura");
    }
}
```

Ejemplo




```java
class Pedido {
    public double calcularTotal() {
        return 100;
    }
}

class PedidoRepository {
    public void guardar(Pedido pedido) {
        System.out.println("Guardando en DB");
    }
}

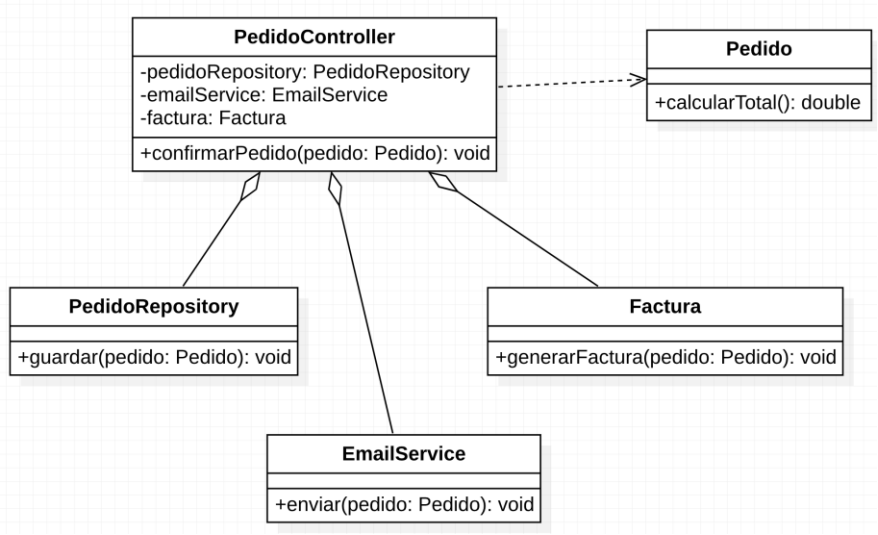
class EmailService {
    public void enviar(Pedido pedido) {
        System.out.println("Enviando email");
    }
}

class Factura {
    public void generarFactura(Pedido pedido) {
        System.out.println("Generando factura");
    }
}
```

Controller

- **Problema**: ¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?
- **Solución**: Asignar la responsabilidad de un mensaje de evento del sistema a una clase que representa una de las siguientes opciones:
  - Representa el sistema global, dispositivo o subsistema.
  - Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo llamado *<NombreDelCasoDeUso>Controller*.
  - Por cada clase del dominio del sistema que requiera recibir o enviar datos a la interfaz, cree su propio controlador. *<Entity>Controller*

Ejemplo




```java
class PedidoController {
    private PedidoRepository pedidoRepository;
    private EmailService emailService;
    private FacturaService facturaService;

    public PedidoController(PedidoRepository pedidoRepository,
                            EmailService emailService,
                            FacturaService facturaService) {
        this.pedidoRepository = pedidoRepository;
        this.emailService = emailService;
        this.facturaService = facturaService;
    }

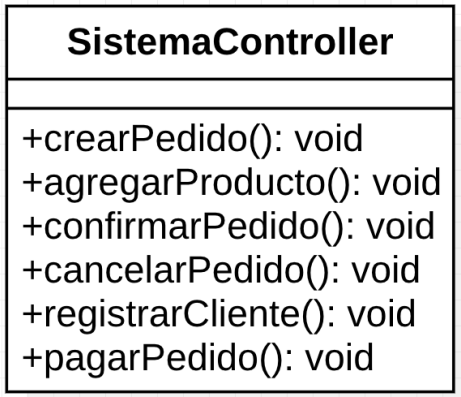
    public void confirmarPedido(Pedido pedido) {
        pedido.confirmar();
        pedidoRepository.guardar(pedido);
        emailService.enviarConfirmacion(pedido);
        facturaService.generar(pedido);
    }
}
```

Tipos de controllers

- Hay **3 tipos**:
  - Controller Global (Sistema)
  - Controller por Caso de Uso (Funcionalidad)
  - Controller por Entidad

Controller Global (Sistema)

- Un **único controlador para todo el sistema.**
- Se transforma en:
  - God Controller
  - clase gigante
  - muchas responsabilidades
  - difícil de mantener

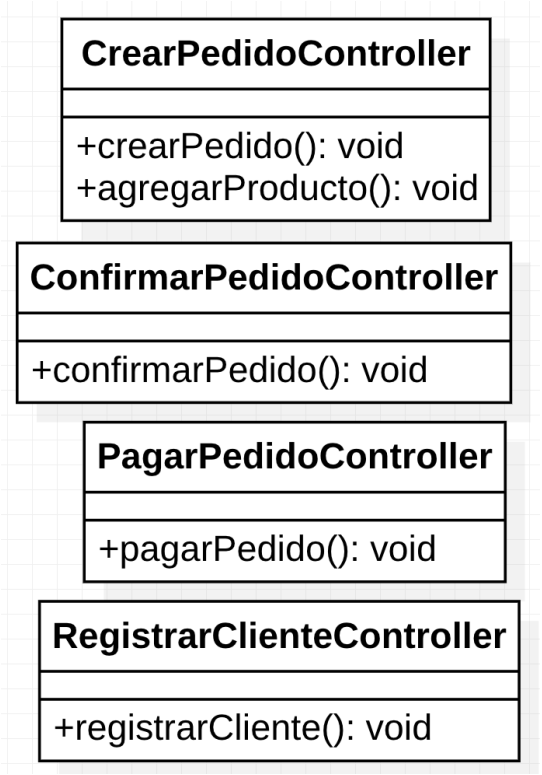


Controller por Caso de Uso

- Un controller por funcionalidad.

Ejemplo:

- CrearPedidoController
- ConfirmarPedidoController
- PagarPedidoController
- RegistrarClienteController



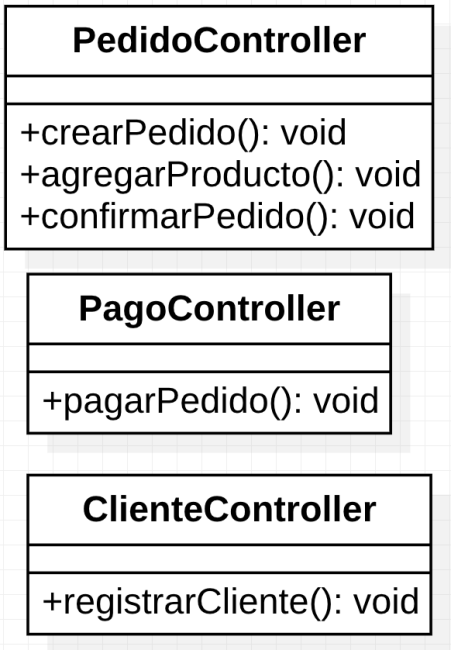
Controller por Entidad

- Cada entidad tiene su controller

Ejemplo:

- PedidoController
- ProductoController
- ClienteController

- Ahora cada controller maneja **todo sobre esa entidad**

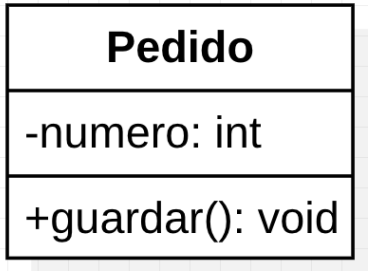


Pure Fabrication

- **Problema**: ¿Quién es el responsable cuando está desesperado y no se quiere violar los principios de alta cohesión y bajo acoplamiento?
- **Solución**: asigne un conjunto altamente cohesivo de responsabilidades a una clase de "comportamiento" artificial o de conveniencia que no representa un concepto del dominio del sistema (algo inventado), para dar soporte a la alta cohesión, bajo acoplamiento y reutilización.
  - crear una clase "inventada"
  - para reducir acoplamiento
  - para aumentar cohesión

Ejemplo

- Problemas:
  - Pedido sabe SQL
  - mezcla dominio con persistencia
  - baja cohesión
  - alto acoplamiento

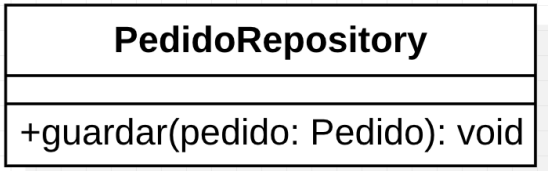



```java
class Pedido {

    public void guardar() {
        System.out.println("INSERT INTO pedido...");
    }
}
```

Ejemplo

- Co pure fabrication, creamos una clase **inventada**
- ¿Por qué se llama "Pure Fabrication"?
  - Pedido existe en el negocio
  - PedidoRepository **NO** existe en el negocio. Es una clase **inventada para el diseño**




```
class PedidoRepository {

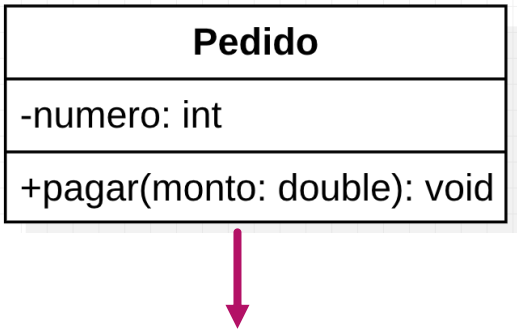
    public void guardar(Pedido pedido) {
        System.out.println("INSERT INTO pedido...");
    }
}
```

Indirection

- **Problema**: ¿Cómo asignar responsabilidades para evitar el acoplamiento directo?
- **Solución**: Asignar la responsabilidad a un objeto intermedio para mediar entre otros componentes o servicios, de manera que no se acoplan directamente.
  - poner una clase en el medio
  - para que dos clases no dependan directamente
  - reducir acoplamiento
- **Beneficios**: permite
  - desacoplar clases
  - cambiar implementaciones
  - centralizar lógica
  - facilitar testing
  - agregar comportamiento

Ejemplo

- Problemas:
  - Pedido depende de MercadoPago
  - difícil cambiar
  - difícil agregar otros pagos
  - alto acoplamiento



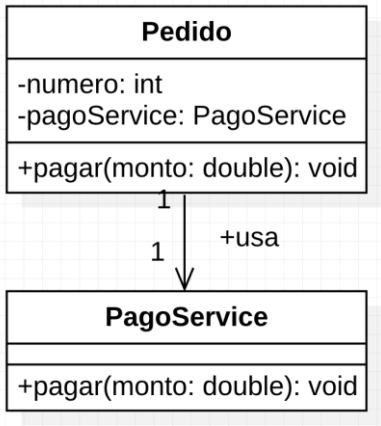

```
class Pedido {

    public void pagar(double monto) {
        MercadoPago mp = new MercadoPago();
        mp.pagar(monto);
    }
}
```

Ejemplo

Ahora:

- Pedido no conoce MercadoPago
- Pedido desacoplado.
- solo conoce PagoService
- bajo acoplamiento




```java
class Pedido {

    private PagoService pagoService;

    public Pedido(PagoService pagoService) {
        this.pagoService = pagoService;
    }

    public void pagar(double monto) {
        pagoService.pagar(monto);
    }
}

class PagoService {

    public void pagar(double monto) {
        MercadoPago mp = new MercadoPago();
        mp.pagar(monto);
    }
}
```

Protected Variations

- **Problema**: ¿Cómo asignar responsabilidades a los objetos, subsistemas, y sistemas de manera que las variaciones o inestabilidad en estos elementos no influya de manera no deseable en otros elementos?
- **Solución**: Identifique los puntos de variaciones predecibles o inestabilidad; asigne las responsabilidades para crear una "interfaz" estable alrededor de ellos.
  - encapsular lo que puede cambiar
  - usar interfaces
  - evitar que el cambio rompa todo
- Evita esto:
  - si cambia una clase → hay que cambiar muchas
  - código frágil
  - sistema rígido
  - mucho acoplamiento

Ejemplo

- Problemas:
  - if gigante
  - si agrego ApplePay → modificar Pedido
  - rompe Open/Closed
  - alto acoplamiento

Pedido
-numero: int
+pagar(tipo: String, monto: double): void

```
class Pedido {

    public void pagar(String tipo, double monto) {

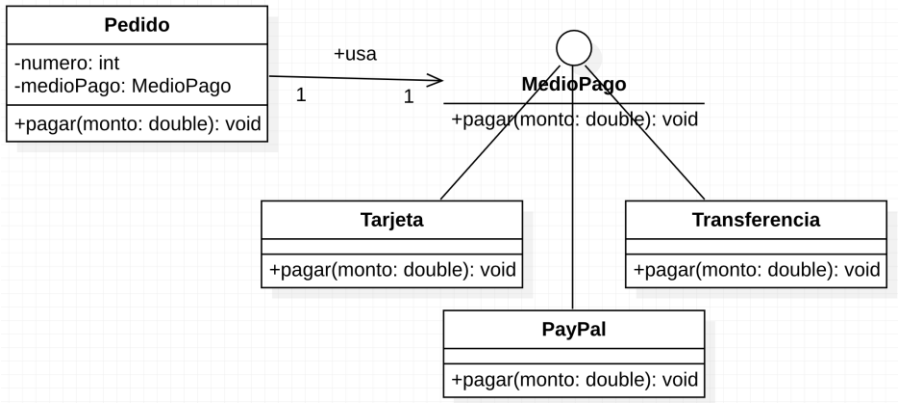
        if(tipo.equals("tarjeta")) {
            Tarjeta t = new Tarjeta();
            t.pagar(monto);
        }

        if(tipo.equals("paypal")) {
            Paypal p = new Paypal();
            p.pagar(monto);
        }

        if(tipo.equals("transferencia")) {
            Transferencia t = new Transferencia();
            t.pagar(monto);
        }
    }
}
```

Ejemplo

- Con "Protected Variations" creamos una abstracción.
- Protegimos:
  - la variación del medio de pago
  - el pedido no cambia
  - solo agrego clases nuevas



Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

